

```
#data source https://www.kaggle.com/datasets/camnugent/sandp500
```

```
from google.colab import files
files.upload()
```

 未选择任何文件 Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving kaggle.json to kaggle.json

```
import os
os.makedirs("/root/.kaggle", exist_ok=True)
!mv kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json
```

```
!kaggle datasets download -d camnugent/sandp500
```

 Dataset URL: <https://www.kaggle.com/datasets/camnugent/sandp500>
License(s): CC0-1.0
Downloading sandp500.zip to /content
0% 0.00/19.3M [00:00<?, ?B/s]
100% 19.3M/19.3M [00:00<00:00, 1.06GB/s]

```
import zipfile
with zipfile.ZipFile("/content/sandp500.zip","r") as stock_price:
    stock_price.extractall("camnugent/sandp500")
```

```
import zipfile
zip_path = "/content/sandp500.zip"
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    file_list = zip_ref.namelist()
    print("The type of ZIP profile: ")
    for file in file_list:
        print(file)
```



```
individual_stocks_5yr/individual_stocks_5yr/WLW_data.csv
individual_stocks_5yr/individual_stocks_5yr/WMB_data.csv
individual_stocks_5yr/individual_stocks_5yr/WMT_data.csv
individual_stocks_5yr/individual_stocks_5yr/WM_data.csv
individual_stocks_5yr/individual_stocks_5yr/WRK_data.csv
individual_stocks_5yr/individual_stocks_5yr/WU_data.csv
individual_stocks_5yr/individual_stocks_5yr/WYNN_data.csv
individual_stocks_5yr/individual_stocks_5yr/WYN_data.csv
individual_stocks_5yr/individual_stocks_5yr/WY_data.csv
individual_stocks_5yr/individual_stocks_5yr/XEC_data.csv
individual_stocks_5yr/individual_stocks_5yr/XEL_data.csv
individual_stocks_5yr/individual_stocks_5yr/XLNX_data.csv
individual_stocks_5yr/individual_stocks_5yr/XL_data.csv
individual_stocks_5yr/individual_stocks_5yr/XOM_data.csv
individual_stocks_5yr/individual_stocks_5yr/XRAY_data.csv
individual_stocks_5yr/individual_stocks_5yr/XRX_data.csv
individual_stocks_5yr/individual_stocks_5yr/XYL_data.csv
individual_stocks_5yr/individual_stocks_5yr/YUM_data.csv
individual_stocks_5yr/individual_stocks_5yr/ZBH_data.csv
individual_stocks_5yr/individual_stocks_5yr/ZION_data.csv
individual_stocks_5yr/individual_stocks_5yr/ZTS_data.csv
merge.sh

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, roc_curve, precision_recall_curve, average_precision_score
from scipy.stats import multivariate_normal
from tabulate import tabulate
import io
import warnings
warnings.filterwarnings("ignore")

stock_price = pd.read_csv("/content/camnugent/sandp500/all_stocks_5yr.csv")
stock_price.head()
```



		date	open	high	low	close	volume	Name
0		2013-02-08	15.07	15.12	14.63	14.75	8407500	AAL
1		2013-02-11	14.89	15.01	14.26	14.46	8882000	AAL
2		2013-02-12	14.45	14.51	14.10	14.27	8126000	AAL
3		2013-02-13	14.30	14.94	14.25	14.66	10259500	AAL

```
stock_price.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 619040 entries, 0 to 619039
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   date        619040 non-null object
1   open        619029 non-null float64
2   high        619032 non-null float64
3   low         619032 non-null float64
4   close       619040 non-null float64
5   volume      619040 non-null int64
6   Name        619040 non-null object
dtypes: float64(4), int64(1), object(2)
memory usage: 33.1+ MB
```

```
stock_price.describe()
```



	open	high	low	close	volume
count	619029.000000	619032.000000	619032.000000	619040.000000	6.190400e+05
mean	83.023334	83.778311	82.256096	83.043763	4.321823e+06
std	97.378769	98.207519	96.507421	97.389748	8.693610e+06
min	1.620000	1.690000	1.500000	1.590000	0.000000e+00
25%	40.220000	40.620000	39.830000	40.245000	1.070320e+06
50%	62.590000	63.150000	62.020000	62.620000	2.082094e+06
75%	94.370000	95.180000	93.540000	94.410000	4.284509e+06

```
print("Column Name: ", stock_price.columns.tolist())
if 'Name' in stock_price.columns:
    unique_names = stock_price['Name'].dropna().unique()
    print(f"\n Totally {len(unique_names)} different Name: ")
    for name in unique_names:
        print(name)
else:
    print("Please check again")
```



ALGN
ALK
ALLE
ALL
ALXN
AMAT
AMD
AME
AMGN
AMG
AMP
AMT
AMZN
ANDV
ANSS
ANTM
AON
AOS
APA
APC
APD
APH

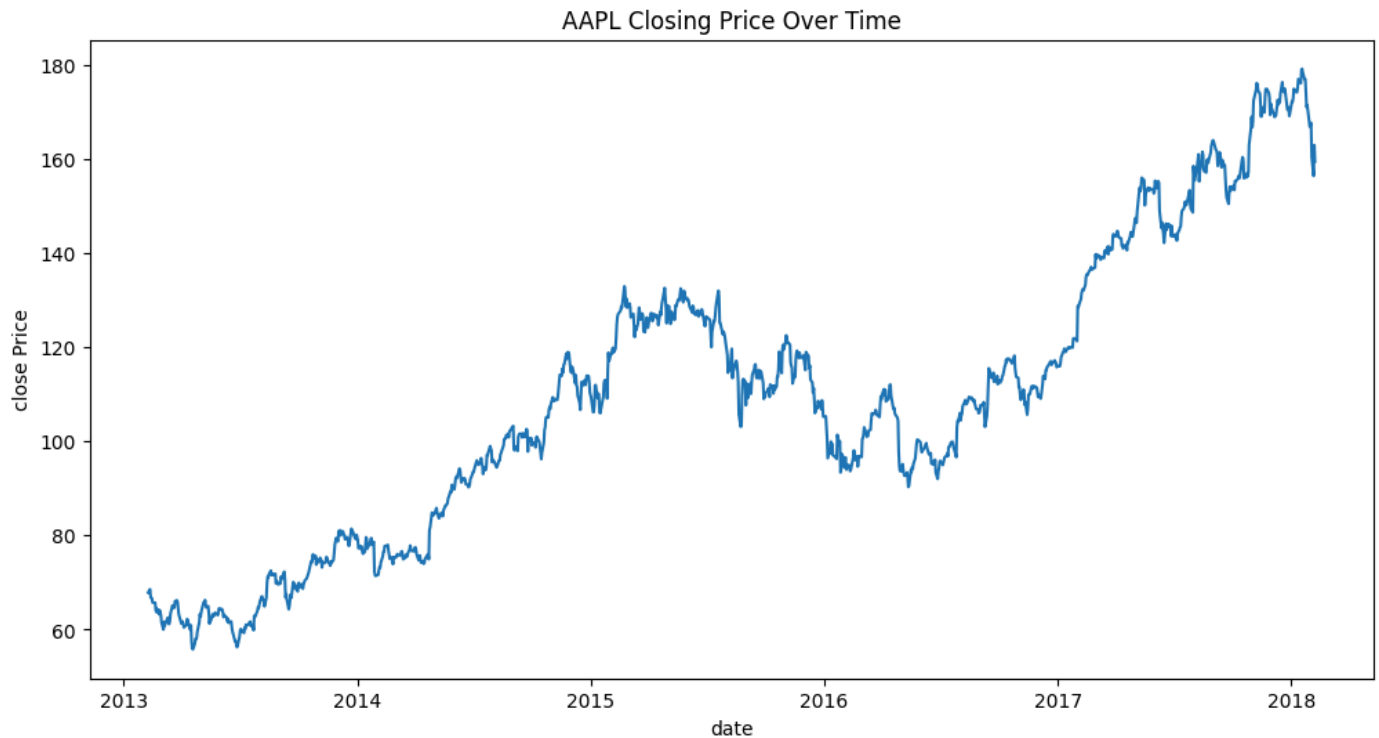
BSX
BWA
BXP
CAG
CAH
CAT
CA
CRG

```
print(stock_price.columns.tolist())
aapl = stock_price[stock_price['Name'] == 'AAPL'].copy()
aapl['date'] = pd.to_datetime(aapl['date'])
aapl.sort_values('date', inplace=True)
aapl.set_index('date', inplace=True)
```

```
prices = aapl['close']
```

```
plt.figure(figsize=(12,6))
plt.plot(prices)
plt.title('AAPL Closing Price Over Time')
plt.xlabel("Date")
plt.ylabel('Close Price')
plt.show()
```

```
↔ ['date', 'open', 'high', 'low', 'close', 'volume', 'Name']
```

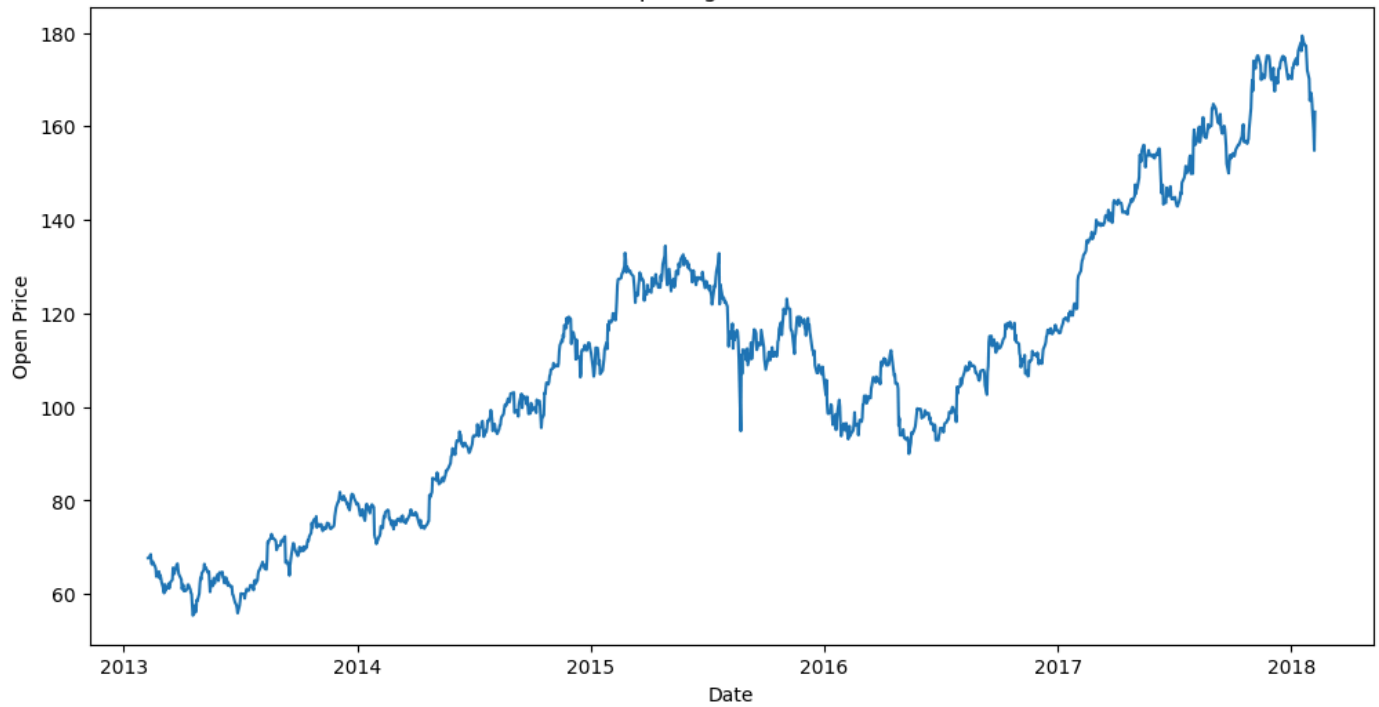


```
prices = aapl['open']
```

```
plt.figure(figsize=(12,6))
plt.plot(prices)
plt.title('AAPL Opening Price Over Time')
plt.xlabel("Date")
plt.ylabel('Open Price')
plt.show()
```



AAPL Opening Price Over Time

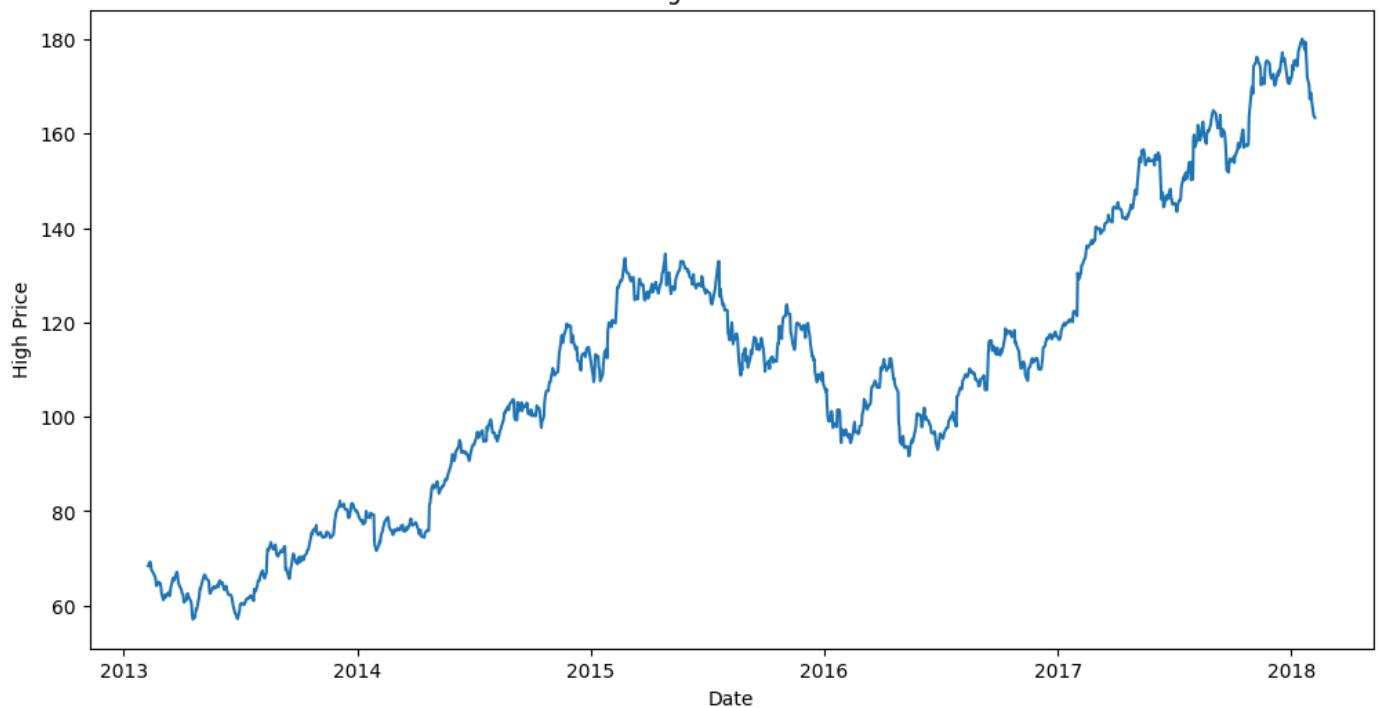


```
prices = aapl['high']

plt.figure(figsize=(12,6))
plt.plot(prices)
plt.title('AAPL High Price Over Time')
plt.xlabel("Date")
plt.ylabel('High Price')
plt.show()
```

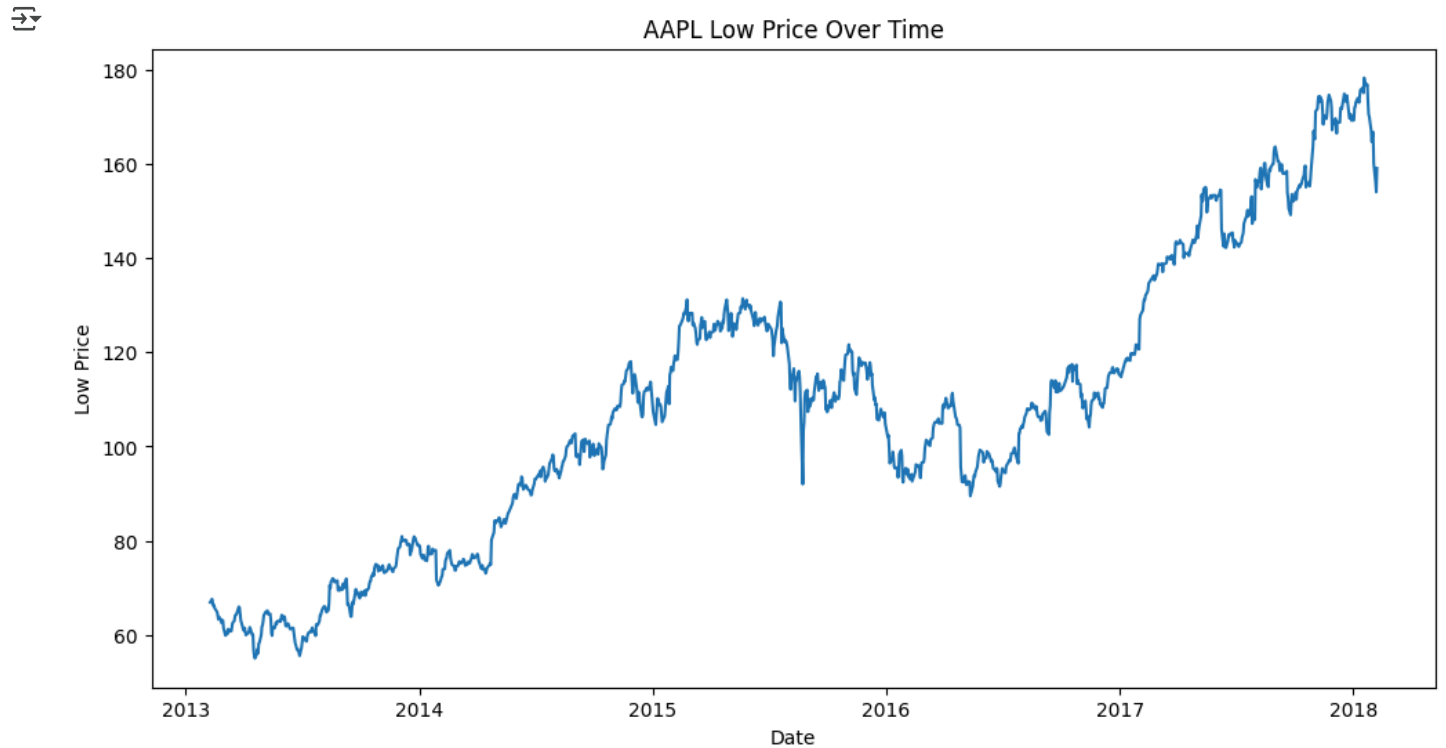


AAPL High Price Over Time



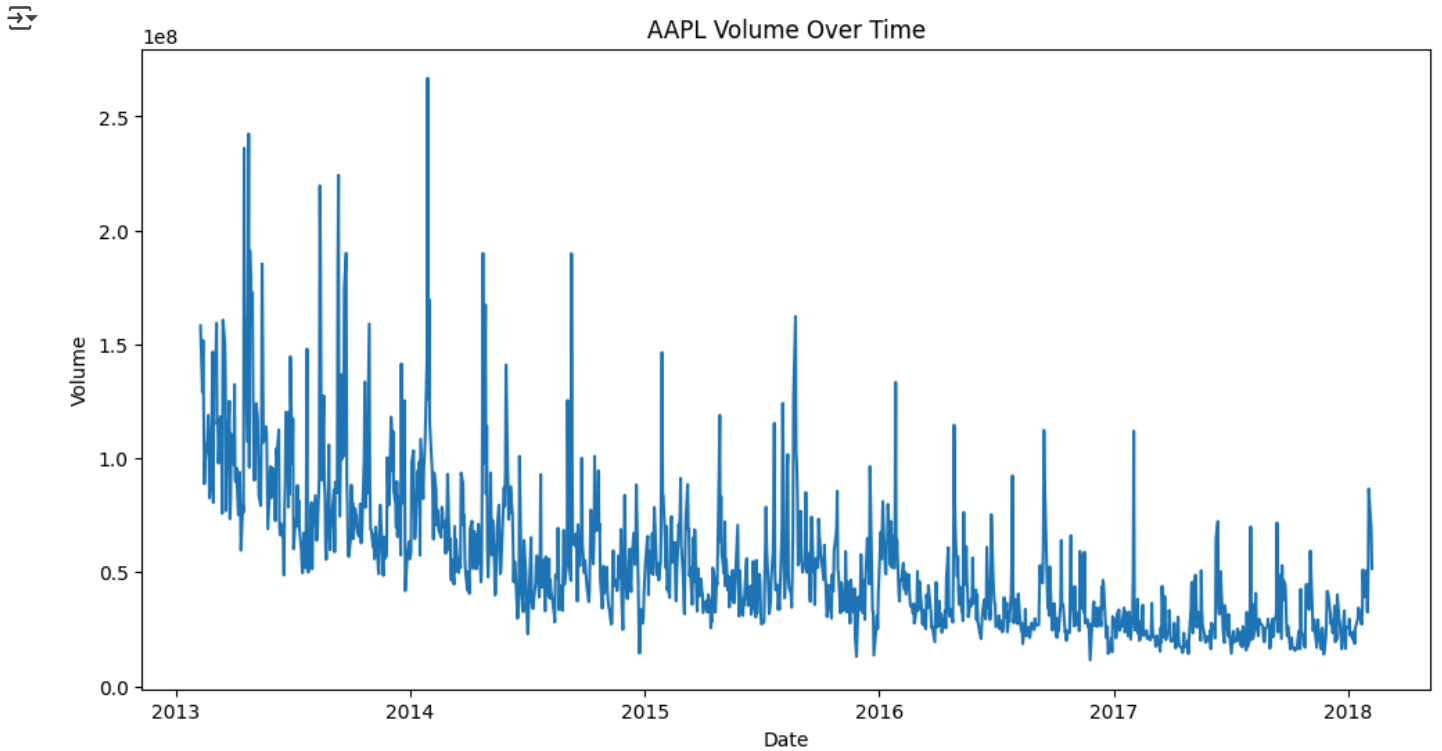
```
prices = aapl['low']
```

```
plt.figure(figsize=(12,6))
plt.plot(prices)
plt.title('AAPL Low Price Over Time')
plt.xlabel("Date")
plt.ylabel('Low Price')
plt.show()
```



```
prices = aapl['volume']

plt.figure(figsize=(12,6))
plt.plot(prices)
plt.title('AAPL Volume Over Time')
plt.xlabel("Date")
plt.ylabel('Volume')
plt.show()
```



```
prices = prices.fillna(method='ffill')
split = int(len(prices)*0.8)
train = prices[:split]
test = prices[split:]
```

```
from statsmodels.tsa.arima.model import ARIMA
```

```
model = ARIMA(train, order=(20,2,1))
model_fit = model.fit()
print(model_fit.summary())
```

```

/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been
self._init_dates(dates, freq)
/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been
self._init_dates(dates, freq)
/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been
self._init_dates(dates, freq)
/usr/local/lib/python3.11/dist-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optim
warnings.warn("Maximum Likelihood optimization failed to "
```

SARIMAX Results

```

=====
Dep. Variable:          close      No. Observations:          1007
Model:                ARIMA(20, 2, 1)  Log Likelihood          -1823.655
Date:                 Mon, 23 Jun 2025  AIC              3691.310
Time:                 02:15:53          BIC              3799.390
Sample:              0                HQIC              3732.378
                             - 1007
```

```
Covariance Type:          opg
```

```

=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1         0.0267      0.028         0.957      0.339      -0.028      0.081
ar.L2        -0.0110      0.030        -0.370      0.711      -0.070      0.047
ar.L3        -0.0067      0.028        -0.239      0.811      -0.061      0.048
ar.L4        -0.0199      0.031        -0.638      0.523      -0.081      0.041
ar.L5        -0.0140      0.029        -0.479      0.632      -0.071      0.043
ar.L6        -0.0035      0.033        -0.107      0.915      -0.068      0.061
ar.L7         0.0551      0.030         1.851      0.064      -0.003      0.113
ar.L8        -0.0258      0.027        -0.952      0.341      -0.079      0.027
ar.L9         0.0302      0.029         1.044      0.296      -0.026      0.087
ar.L10       -0.0092      0.031        -0.300      0.764      -0.069      0.051
ar.L11       -0.0258      0.028        -0.910      0.363      -0.082      0.030
ar.L12       -0.0040      0.031        -0.128      0.898      -0.065      0.057
=====
```

ar.L13	-0.0028	0.028	-0.100	0.920	-0.058	0.052
ar.L14	0.0349	0.031	1.115	0.265	-0.026	0.096
ar.L15	0.0002	0.031	0.008	0.994	-0.060	0.060
ar.L16	0.0008	0.030	0.028	0.977	-0.057	0.059
ar.L17	0.0219	0.029	0.753	0.451	-0.035	0.079
ar.L18	-0.0379	0.030	-1.273	0.203	-0.096	0.020
ar.L19	0.0153	0.033	0.468	0.640	-0.049	0.079
ar.L20	0.0565	0.032	1.758	0.079	-0.006	0.120
ma.L1	-0.9992	0.011	-89.068	0.000	-1.021	-0.977
sigma2	2.1897	0.070	31.194	0.000	2.052	2.327

Ljung-Box (L1) (Q):	0.00	Jarque-Bera (JB):	472.62
Prob(Q):	0.99	Prob(JB):	0.00
Heteroskedasticity (H):	2.12	Skew:	-0.11
Prob(H) (two-sided):	0.00	Kurtosis:	6.35

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```

aapl_10_20 = aapl.loc['2010-01-01':'2020-12-31']
prices = aapl_10_20['close']

start = len(train)
end = len(train) + len(test) - 1
pred = model_fit.predict(start=start, end=end, typ='levels')

plt.figure(figsize=(12,6))
plt.plot(train.index, train, label='Train')
plt.plot(test.index, test, label='Test')
plt.plot(test.index, pred, label='Predicted')
plt.title('AAPL Closing Price Prediction (2010-2020)')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.show()

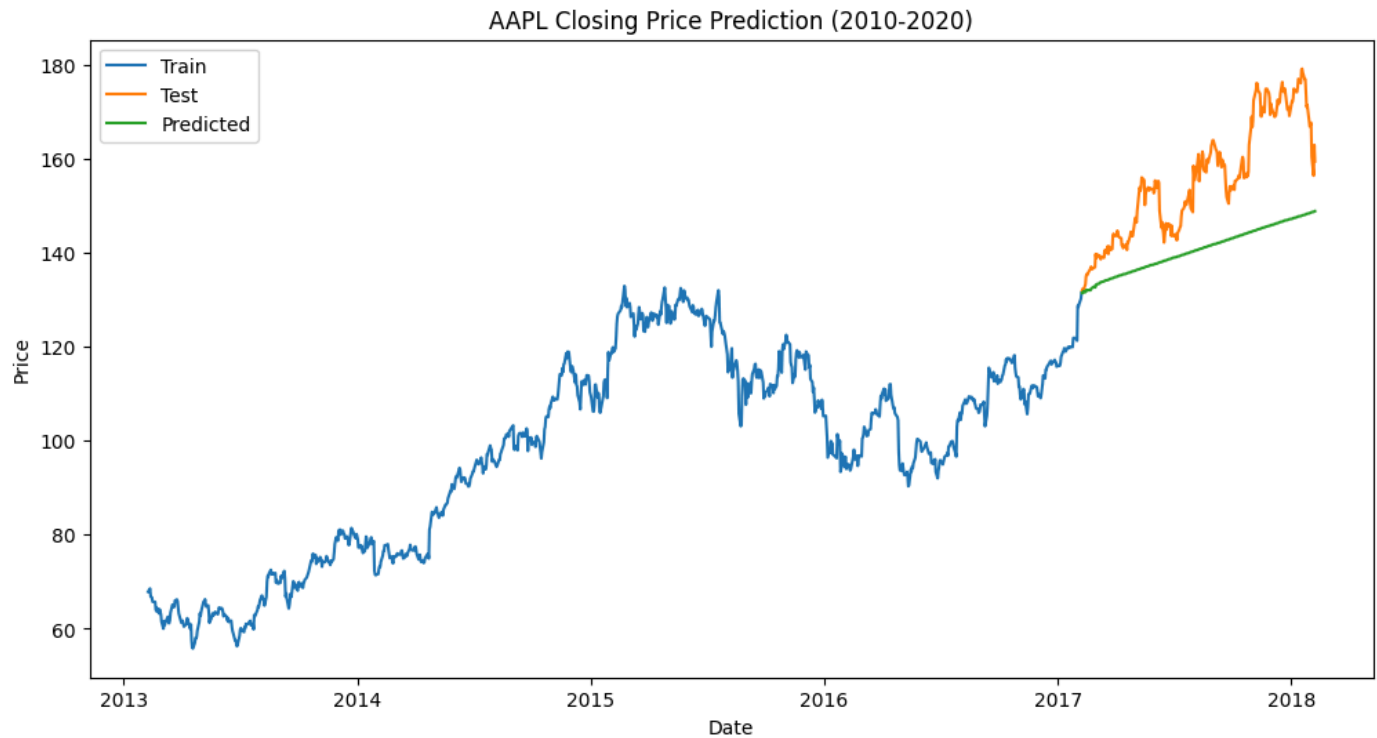
mse = mean_squared_error(test, pred)
print(f'MSE: {mse:.4f}')

```

```

/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is
return get_prediction_index(

```



```

n_steps = 20
forecast = model_fit.forecast(steps=n_steps)
print(forecast)

```

```

1007    131.500475
1008    131.512744
1009    131.864591
1010    131.670758
1011    132.103853
1012    132.149353
1013    132.054352
1014    132.130486
1015    132.101421
1016    132.406666
1017    132.512140
1018    132.637996
1019    132.887124
1020    132.681112
1021    132.851228
1022    133.315195
1023    133.387731
1024    133.436136
1025    133.608152
1026    133.750730

```

Name: predicted_mean, dtype: float64

```

/usr/local/lib/python3.11/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is
return get_prediction_index(

```

#LSTM

```

from sklearn.preprocessing import MinMaxScaler
from keras.models import Sequential
from keras.layers import Dense, LSTM, Dropout

```

```

scaler = MinMaxScaler(feature_range=(0, 1))
scaled_prices = scaler.fit_transform(prices.values.reshape(-1,1))

```

```
def create_dataset(data, time_step=60):
    x, y = [], []
    for i in range(len(data)-time_step):
        x.append(data[i:i+time_step, 0])
        y.append(data[i+time_step, 0])
    return np.array(x), np.array(y)

time_step = 60
x, y = create_dataset(scaled_prices, time_step)

x = x.reshape(X.shape[0], x.shape[1], 1)

train_size = int(len(X)*0.8)
X_train, X_test = X[:train_size], X[train_size:]
Y_train, Y_test = Y[:train_size], Y[train_size:]

#Establish LSTM Model
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(time_step,1)))
model.add(Dropout(0.2))
model.add(LSTM(units=50, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(units=1))

model.compile(optimizer='adam', loss='mean_squared_error')
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 50)	10,400
dropout (Dropout)	(None, 60, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_1 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

Total params: 30,651 (119.73 KB)
Trainable params: 30,651 (119.73 KB)

```
history = model.fit(X_train, Y_train, epochs=50, batch_size=32, validation_data=(X_test, Y_test))
```



Epoch 35/50

30/30  2s 49ms/step - loss: 0.0012 - val_loss: 0.0025

Epoch 36/50

30/30  4s 103ms/step - loss: 0.0014 - val_loss: 0.0018

Epoch 37/50

30/30  3s 49ms/step - loss: 0.0011 - val_loss: 0.0011

Epoch 38/50

30/30  3s 50ms/step - loss: 0.0011 - val_loss: 0.0018

Epoch 39/50

30/30  3s 50ms/step - loss: 0.0012 - val_loss: 7.1278e-04

Epoch 40/50

30/30  4s 83ms/step - loss: 0.0010 - val_loss: 0.0015

Epoch 41/50

30/30  2s 52ms/step - loss: 0.0011 - val_loss: 0.0016

Epoch 42/50

30/30  3s 69ms/step - loss: 0.0012 - val_loss: 0.0015

Epoch 43/50

30/30  2s 51ms/step - loss: 0.0013 - val_loss: 0.0037

Epoch 44/50

30/30  1s 49ms/step - loss: 0.0011 - val_loss: 8.0952e-04

Epoch 45/50

30/30  3s 50ms/step - loss: 0.0011 - val_loss: 6.3693e-04

Epoch 46/50

30/30  4s 85ms/step - loss: 0.0011 - val_loss: 0.0030

Epoch 47/50

30/30  2s 50ms/step - loss: 0.0012 - val_loss: 6.4715e-04

Epoch 48/50

30/30  3s 49ms/step - loss: 0.0012 - val_loss: 0.0016

Epoch 49/50

30/30  2s 50ms/step - loss: 8.9260e-04 - val_loss: 8.6090e-04

Epoch 50/50

30/30  1s 49ms/step - loss: 9.8430e-04 - val_loss: 0.0015

train_predict = model.predict(X_train)

test_predict = model.predict(X_test)

train_predict = scaler.inverse_transform(train_predict)

Y_train_actual = scaler.inverse_transform(Y_train.reshape(-1,1))

test_predict = scaler.inverse_transform(test_predict)

Y_test_actual = scaler.inverse_transform(Y_test.reshape(-1,1))

 **30/30**  1s 32ms/step
8/8  0s 16ms/step